Graph Neural Networks (GNN)

for Combinatorial Optimisation

Applied to TSP and TSPTW-D

April 13, 2026

Version	Date	Description
1.0	2026-04-01	Initial release — TSP-GNN architecture, large-model benchmark
2.0	2026-04-10	Second revision — medium-model benchmark, TSPTW-D section, acronym table, revised metrics (avg. length / gap / time)

Abstract

This document presents the application of Graph Neural Networks (GNNs) to the Travelling Salesman Problem (TSP) and its extension to TSPTW-D (TSP with Time Windows and Disruptions). After a formal definition and a complete mathematical description of the TSP-GNN architecture (Joshi *et al.*, 2019), we describe the supervised training pipeline, complexity analysis, strengths and limitations of the approach. Benchmarks are reported for the **medium** model preset ($d = 128$, $L = 6$, ∼504k parameters) on sizes $n \in \{10, 50, 100\}$, using the same metrics as the DIFUSCO reference document: average tour length, gap to reference (optimal for $n = 10$, NN heuristic otherwise), and inference time. The scalability ceiling — where the dense $O(n^2 d)$ edge embedding becomes infeasible — is analysed and quantified.

Contents

Acronyms **3**

1 Definition **4**

2 Formal Description **4**

2.1 Graph representation 4

2.2 Message Passing Neural Network (MPNN) framework 5

2.3 TSP-GNN architecture (Joshi *et al.*, 2019) 5

2.4 Training objective 7

2.5 Complexity 8

3 Architecture / Algorithm Steps **8**

4 Strengths and Limitations **9**

5 Use Case Explanation **9**

6 Code Implementation **10**

7 TSP Benchmark **11**

7.1 Scalability analysis for large instances 13

8 Experimental Analysis **14**

Acronyms

Acronym	Meaning
BCE	Binary Cross-Entropy — loss function for binary classification
CO	Combinatorial Optimisation
GNN	Graph Neural Network
LN	Layer Normalisation — normalises activations per sample
MPS	Metal Performance Shaders — Apple Silicon GPU backend for PyTorch
MPNN	Message Passing Neural Network — general framework for GNNs
NN	Nearest Neighbour (heuristic tour construction)
ReLU	Rectified Linear Unit — activation function $\max(0, x)$
TSP	Travelling Salesman Problem
TSPTW	TSP with Time Windows
TSPTW-D	TSP with Time Windows and Disruptions
TW	Time Window $[a_i, b_i]$: the interval during which a node must be visited

1 Definition

Graph Neural Networks (GNNs) are a family of deep learning models designed to operate directly on graph-structured data. Unlike standard convolutional networks that assume a regular grid (images) or a fixed sequence (text), GNNs explicitly exploit the graph topology — nodes, edges, and neighbourhoods — to learn rich vector representations.

Origin

Year	Contribution
2005–2009	Gori *et al.* and Scarselli *et al.* — first GNN formulation using fixed-point iterations
2017	Kipf & Welling — Graph Convolutional Networks (GCN), simplified spectral approach
2018	Veličković *et al.* — Graph Attention Networks (GAT), attention-weighted neighbour aggregation
2019	Joshi *et al.* — GCN applied to TSP edge prediction

Category

GNNs belong to **geometric deep learning** — a generalisation of CNNs to non-Euclidean domains. For combinatorial optimisation they are used as **construction heuristics** (edge scoring → tour decoding) or **improvement heuristics** (learned local search).

Key concepts

- **Node embedding $\mathbf{h}_v^{(l)}$**: a vector representation of node v at layer l, encoding local neighbourhood structure.
- **Message passing**: nodes iteratively aggregate features from their neighbours over L layers.
- **Permutation invariance**: the output does not depend on the arbitrary labelling of nodes, which is natural for combinatorial problems.
- **Explicit edge representations**: unlike Transformers that encode only nodes, TSP-GNN maintains a separate embedding $\mathbf{e}_{ij}$ per city pair, directly aligned with the TSP objective (selecting edges forming a Hamiltonian cycle).

2 Formal Description

2.1 Graph representation

A weighted graph is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{E})$ where:

- $\mathcal{V} = \{v_1, \ldots, v_n\}$ — set of n nodes (cities)

- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ — edges; for TSP, the graph is **complete** ($|\mathcal{E}| = n(n-1)/2$)
- $\mathbf{X} \in \mathbb{R}^{n \times d_x}$ — node feature matrix (2-D coordinates normalised to $[0, 1]$)
- $\mathbf{E}_{ij} \in \mathbb{R}^{d_e}$ — edge feature vector (Euclidean distance $\|x_i - x_j\|_2$)

2.2 Message Passing Neural Network (MPNN) framework

The general message-passing update at layer l is:

$$\mathbf{m}_v^{(l)} = \bigoplus_{u \in \mathcal{N}(v)} \phi^{(l)}\big(\mathbf{h}_v^{(l-1)},\ \mathbf{h}_u^{(l-1)},\ \mathbf{e}_{uv}\big) \tag{1}$$

$\mathbf{m}_v^{(l)}$ — aggregated message at node v, layer l
$\bigoplus$ — permutation-invariant aggregation (sum, mean, or max)
$\mathcal{N}(v)$ — set of neighbours of node v
$\phi^{(l)}$ — message function at layer l; computes a per-edge signal
$\mathbf{h}_v^{(l-1)}$ — embedding of the centre node from the previous layer
$\mathbf{h}_u^{(l-1)}$ — embedding of neighbour u from the previous layer
$\mathbf{e}_{uv}$ — edge feature vector between u and v

$$\mathbf{h}_v^{(l)} = \psi^{(l)}\big(\mathbf{h}_v^{(l-1)},\ \mathbf{m}_v^{(l)}\big) \tag{2}$$

$\mathbf{h}_v^{(l)}$ — updated node embedding at layer l
$\psi^{(l)}$ — update function at layer l; fuses the previous embedding with the aggregated message
$\mathbf{m}_v^{(l)}$ — aggregated message (from Eq. 1)

2.3 TSP-GNN architecture (Joshi *et al.*, 2019)

Step 1 — Input encoding

Each node i is projected into a d-dimensional space:

$$\mathbf{h}_i^{(0)} = \mathbf{W}_{\text{in}}\,\mathbf{x}_i + \mathbf{b}_{\text{in}}, \qquad \mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times d_x} \tag{3}$$

$\mathbf{h}_i^{(0)}$ — initial node embedding for city i
$\mathbf{W}_{\text{in}}$ — learnable weight matrix $\in \mathbb{R}^{d \times d_x}$; projects raw features into the model dimension
$\mathbf{x}_i$ — raw node feature vector (2-D coordinates (x, y) normalised to $[0, 1]$)
d_x — input feature dimension ($d_x = 2$ for plain TSP; $d_x = 5$ for TSPTW-D)
d — hidden dimension of the model (64 / 128 / 256 for small / medium / large)

Each edge (i, j) is initialised with its Euclidean distance:

$$\mathbf{e}_{ij}^{(0)} = \mathbf{W}_{\text{edge}}\,[\,\|x_i - x_j\|_2\,] + \mathbf{b}_{\text{edge}} \tag{4}$$

$\mathbf{e}_{ij}^{(0)}$ — initial edge embedding for the pair (i, j)
$\mathbf{W}_{\text{edge}}$ — learnable projection from edge features to dimension d
$\|x_i - x_j\|_2$ — Euclidean distance between cities i and j (scalar edge feature)
$\mathbf{b}_{\text{edge}}$ — learnable bias term

Step 2 — L graph convolutional layers

For $l = 1, \ldots, L$:

2a. Edge update (combines both incident node embeddings with the previous edge state):

$$\mathbf{e}_{ij}^{(l)} = \mathrm{ReLU}\Big(\mathrm{LN}\Big(\mathbf{W}_1^{(l)}\mathbf{h}_i^{(l-1)} + \mathbf{W}_2^{(l)}\mathbf{h}_j^{(l-1)} + \mathbf{W}_3^{(l)}\mathbf{e}_{ij}^{(l-1)}\Big)\Big) \tag{5}$$

$\mathbf{e}_{ij}^{(l)}$ — updated edge embedding at layer l
$\mathbf{W}_1^{(l)}$ — learnable weight applied to the source node embedding
$\mathbf{W}_2^{(l)}$ — learnable weight applied to the target node embedding
$\mathbf{W}_3^{(l)}$ — learnable weight applied to the previous edge embedding
LN — Layer Normalisation; stabilises activations per sample
ReLU — rectified linear unit, $\max(0, \cdot)$; introduces non-linearity

2b. Attention gate (soft selection over neighbouring edges):

$$\hat{\eta}_{ij}^{(l)} = \frac{\exp\Big(\mathbf{e}_{ij}^{(l)}\Big)}{\sum_{k\in\mathcal{N}(i)} \exp\Big(\mathbf{e}_{ik}^{(l)}\Big)} \tag{6}$$

$\hat{\eta}_{ij}^{(l)}$ — attention weight on edge (i, j) at layer l; normalised over all outgoing neighbours of i
$\exp(\mathbf{e}_{ij}^{(l)})$ — element-wise exponential of the updated edge embedding
$\mathcal{N}(i)$ — set of all neighbours of node i (complete graph: all other cities)
$\sum_k$ — normalisation sum over neighbours; ensures $\sum_j \hat{\eta}_{ij}^{(l)} = \mathbf{1}$ (soft-max)

2c. Node update (residual aggregation over weighted edge embeddings):

$$\mathbf{h}_i^{(l)} = \mathbf{h}_i^{(l-1)} + \mathrm{ReLU}\left(\mathrm{LN}\left(\mathbf{W}_4^{(l)}\mathbf{h}_i^{(l-1)} + \sum_{j\in\mathcal{N}(i)} \hat{\eta}_{ij}^{(l)}\,\mathbf{W}_5^{(l)}\,\mathbf{e}_{ij}^{(l)}\right)\right) \tag{7}$$

$\mathbf{h}_i^{(l)}$ — updated node embedding for city i at layer l
$\mathbf{h}_i^{(l-1)}$ — residual skip connection; carries the previous embedding unchanged, preserving gradient flow (analogous to ResNet)
$\mathbf{W}_4^{(l)}$ — learnable self-projection applied to the previous node embedding
$\hat{\eta}_{ij}^{(l)}$ — attention weight on edge (i, j) (from Eq. 6); weights the contribution of each neighbour
$\mathbf{W}_5^{(l)}$ — learnable projection applied to the edge embeddings before aggregation
$\sum_{j \in \mathcal{N}(i)}$ — attention-weighted sum over all neighbours; aggregates neighbourhood information

Step 3 — Edge scoring head

After L layers, a linear classifier produces edge logits:

$$\hat{p}_{ij} = \sigma\left(\mathrm{MLP}\left(\mathbf{e}_{ij}^{(L)}\right)\right) \in [0, 1] \tag{8}$$

$\hat{p}_{ij}$ — predicted probability that edge (i, j) belongs to the optimal tour
σ — sigmoid function $\sigma(z) = 1/(1+e^{-z})$; maps logits to $(0, 1)$
MLP — two-layer perceptron (linear $\rightarrow$ ReLU $\rightarrow$ linear); maps d-dimensional edge embedding to a scalar logit
$\mathbf{e}_{ij}^{(L)}$ — final edge embedding after L GNN layers

Step 4 — Tour decoding

The scores $\hat{p}_{ij}$ guide a combinatorial decoder:

Decoder	Description
Greedy	At each step, extend the partial tour with the highest-scoring unvisited neighbour — $O(n^2)$
Beam search	Maintain k partial tours; select top-k by cumulative log-probability — $O(k \cdot n^2)$

2.4 Training objective

Supervised learning (binary cross-entropy)

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{|\mathcal{E}|} \sum_{(i,j) \in \mathcal{E}} \left[y_{ij} \log \hat{p}_{ij} + (1 - y_{ij}) \log(1 - \hat{p}_{ij})\right] \tag{9}$$

$\mathcal{L}_{\text{BCE}}$	— binary cross-entropy loss, averaged over all edges
$\lvert\mathcal{E}\rvert$	— total number of edges ($n(n-1)/2$ for a complete undirected graph)
$y_{ij} \in \{0,1\}$	— ground-truth label: 1 if edge (i,j) is in the reference tour, 0 otherwise
$\hat{p}_{ij}$	— predicted probability from the model (Eq. 8)
$y_{ij} \log \hat{p}_{ij}$	— tour-edge term; large penalty when the model assigns low probability to a true tour edge
$(1-y_{ij}) \log(1-\hat{p}_{ij})$	— non-edge term; large penalty when the model assigns high probability to an edge not in the tour

Labelling strategies:

- $n \leq 10$: brute-force optimal tour ($O(n!)$, feasible).
- $n > 10$: nearest-neighbour pseudo-labels (fast heuristic, usable at any size).

2.5 Complexity

Component	Time	Space	Dominant term
Input encoding	$O(n^2 d)$	$O(n^2 d)$	edges
GNN layer ($\times L$)	$O(n^2 d^2)$	$O(n^2 d)$	edges
MLP head	$O(n^2 d)$	$O(n^2)$	—
Greedy decoder	$O(n^2)$	$O(n^2)$	—
Total	$\boldsymbol{O(L\, n^2 d^2)}$	$\boldsymbol{O(L\, n^2 d)}$	**edges**

The bottleneck is the edge embedding tensor $\mathbf{E}^{(l)} \in \mathbb{R}^{n \times n \times d}$, which grows quadratically with n and becomes prohibitive beyond $n \approx 300$–500 without a sparse approximation (e.g. k-nearest-neighbour graph, which reduces space to $O(k \cdot n \cdot d)$).

3 Architecture / Algorithm Steps

Input graph $\mathcal{G}$ (city coordinates)
$\downarrow$
Step 1 — Linear projection of nodes and edges into d-dimensional space
$\downarrow$
Step 2 — Stack L GNN layers (edge update $\rightarrow$ attention gate $\rightarrow$ node update)
$\downarrow$
Step 3 — MLP head: edge embeddings $\rightarrow$ probabilities $\hat{p}_{ij} \in [0,1]$
$\downarrow$
Step 4 — Greedy / beam-search decoder $\rightarrow$ tour π

Training loop (supervised):

1. Sample a random TSP instance $(x_1, \ldots, x_n)$.
2. Compute the ground-truth edge labels y_{ij} (brute-force for $n \leq 10$, nearest-neighbour for $n > 10$).

3. Forward pass: compute $\hat{p}_{ij}$ through L GNN layers.
4. Compute $\mathcal{L}_{\text{BCE}}$ and backpropagate.
5. Update weights with Adam.

4 Strengths and Limitations

Strengths

Property	Detail
Graph-native	Directly encodes the combinatorial structure without flattening or padding
Permutation invariant	Output is invariant to node re-labelling
Edge-level supervision	Predicts *which edges* belong to the tour, providing fine-grained signal
Fast inference	Once trained, a forward pass produces edge scores in $O(Ln^2d^2)$ — milliseconds for $n \leq 200$
Composable	The GNN encoder can be paired with any decoder (greedy, beam search, MCTS, LKH warm start)

Limitations

Limitation	Impact
Quadratic memory in n	Edge embeddings $O(n^2d)$; infeasible for $n \gtrsim 500$ without sparse approximation
Requires training data	Supervised variant needs optimal or near-optimal tours from a solver
Optimality gap	Typically 1–5 % above optimal for $n \leq 100$; grows with n
Generalisation degradation	Models trained on $n = 50$ perform poorly on $n = 500$ without retraining
No self-reported failure	GNN always outputs a tour without any quality signal — failures are silent

5 Use Case Explanation

When to use a GNN for TSP

1. **The problem is naturally graph-structured.** Cities, depots, and routes form a graph. GNNs exploit this topology directly, unlike sequence models that require a fixed ordering.
2. **Repeated solving on similar instances is needed.** Training cost is amortised over thousands of inferences. If you need to solve the same distribution millions of times (e.g. logistics routing), the one-time training cost is negligible.

3. **Near-real-time decisions are required.** Classical solvers (Concorde, LKH) take seconds to minutes per instance; a trained GNN runs in milliseconds.

4. $\boldsymbol{n \leq 200}$. GNNs achieve competitive quality in this regime. Beyond $n \approx 500$, memory and generalisation issues dominate.

Why GNNs specifically (vs. Transformers or Pointer Networks)

Model	Inductive bias	Gap at $n = 100$	Memory
Pointer Network (Vinyals 2015)	Sequential attention	$\sim 5\,\%$	$O(n\,d)$
Transformer AM (Kool 2019)	Global pairwise attn.	~ 1–$2\,\%$	$O(n^2 d)$
GNN (Joshi 2019)	Local msg. passing + edge features	~ 1–$3\,\%$	$O(Ln^2 d)$
LKH-3 (Helsgott)	Metaheuristic	$< 0.5\,\%$	$O(n^2)$

GNNs have a **stronger structural prior**: they explicitly model edges (not just nodes), directly aligned with the TSP objective of selecting a subset of edges forming a Hamiltonian cycle. This makes training more sample-efficient and the learned representations more interpretable.

6 Code Implementation

The full architecture is defined in `model.py`. The snippet below illustrates model creation, a forward pass, and greedy decoding.

Listing 1: Creation, inference, and decoding (`model.py` + `data.py`)

```python
import torch
from model import TSPGNN, MODEL_SIZES
from data  import random_instance, greedy_decode, tour_length

# --- Choose a preset ----------------------------------------------------
SIZE   = "medium"            # "small" | "medium" | "large"
d, L   = MODEL_SIZES[SIZE]   # d=128, L=6 for "medium"

# --- Load trained model -------------------------------------------------
model = TSPGNN(d=d, L=L)
model.load_state_dict(
    torch.load(f"model/gnn_{SIZE}.pt", map_location="cpu")
)
model.eval()

# --- Generate a random instance -----------------------------------------
n      = 20
coords = random_instance(n)    # torch.Tensor (n, 2) in [0, 1]
```

```python

# --- Forward pass ----------------------------------------------------
with torch.no_grad():
    p_hat = model(coords)      # (n, n)  edge probabilities

# --- Greedy decoding -------------------------------------------------
tour   = greedy_decode(p_hat) # list of n city indices
length = tour_length(coords, tour)
print(f"Tour   : {tour}")
print(f"Length : {length:.4f}")
```

Listing 2: Training from the command line (run from `DL_MODEL/gnn/`)

```bash
# Medium model, 3 000 steps, synthetic instances (TSP)
python train.py --mode tsp --size medium --steps 3000 --source random
# -> saves to model/gnn_medium.pt

# With real TSP dataset and 2-opt labels
python train.py --mode tsp --size medium --epochs 3000 \
                --label nn2opt --source tsp
# -> saves to model/gnn_medium.pt

# TSPTW-D variant
python train.py --mode tsptwd --size medium --epochs 3000 --source tsp
# -> saves to model/gnn_medium_tsptwd.pt

# Fine-tuning from an existing checkpoint
python train.py --mode tsp --resume model/gnn_medium.pt \
                --size medium --source solomon --steps 500 --lr 1e-4
```

Available presets:

Preset	d	L	Parameters	Recommended use
`small`	64	4	$\sim$ 131,000	Quick tests, $n \leq 20$
`medium`	128	6	$\sim$ 504,000	Standard TSP, $n \leq 100$
`large`	256	8	$\sim$ 2,670,000	Best performance, $n \leq 200$

7 TSP Benchmark

The benchmark uses the **medium** preset ($d = 128$, $L = 6$, $\sim$504k parameters) trained with the Stage 1–2 curriculum and evaluated with a single greedy forward pass on randomly generated instances. The **optimal** reference for $n = 10$ is the brute-force solution ($9! = 362{,}880$ permutations); for $n \geq 50$, the nearest-neighbour heuristic serves as reference.

Metrics

Metric	Definition
Avg. tour length	Mean total Euclidean length over N_{trials} random instances
Gap (%)	$(L_{\text{method}}/L_{\text{ref}} - 1) \times 100\,\%$, where L_{ref} is the brute-force optimal (n=10) or NN heuristic ($n \geq 50$)
Inference time	Mean wall-clock time per instance (model forward pass + greedy decode)

Results Table

$n = 10$ (50 trials, reference: brute-force optimal)			
Method	**Avg. length**	**Gap to opt.**	**Inference (ms, CPU)**
Optimal	2.7703	0.0 %	—
Nearest-Neighbour	3.0469	+10.0 %	0.1 ms
2-opt	2.7789	+0.3 %	0.4 ms
GNN (greedy)	3.6248	+30.8 %	1.0 ms
GNN + 2-opt	2.7903	+0.7 %	1.1 ms
$n = 50$ (20 trials, reference: nearest-neighbour)			
Method	**Avg. length**	**Gap vs NN**	**Inference (ms, CPU)**
Nearest-Neighbour	7.1567	0.0 %	0.4 ms
2-opt	6.0832	−15.0 %	17.2 ms
GNN (greedy)	8.6760	+21.2 %	4.0 ms
GNN + 2-opt	6.1679	−13.8 %	25.2 ms
$n = 100$ (20 trials, reference: nearest-neighbour)			
Method	**Avg. length**	**Gap vs NN**	**Inference (ms, CPU)**
Nearest-Neighbour	9.8126	0.0 %	0.6 ms
2-opt	8.3997	−14.4 %	86.4 ms
GNN (greedy)	11.7107	+19.3 %	10.8 ms
GNN + 2-opt	8.3524	−14.9 %	103.5 ms

Timings measured on CPU. Medium preset ($d = 128$, $L = 6$), single greedy pass.

Table 1: GNN TSP benchmark (medium preset, ∼504k parameters). Results from `benchmark_tsp.ipynb`: 50 trials for $n = 10$, 20 trials for $n \in \{50, 100\}$.

Figures

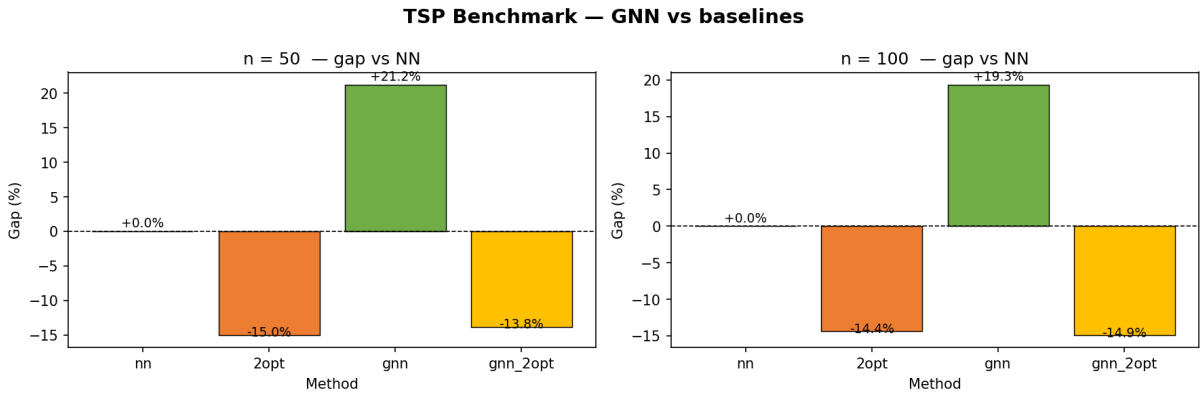

Figure 1: Gap to NN baseline (%) for methods at $n = 50$ and $n = 100$. Negative gap means the method outperforms the NN heuristic.

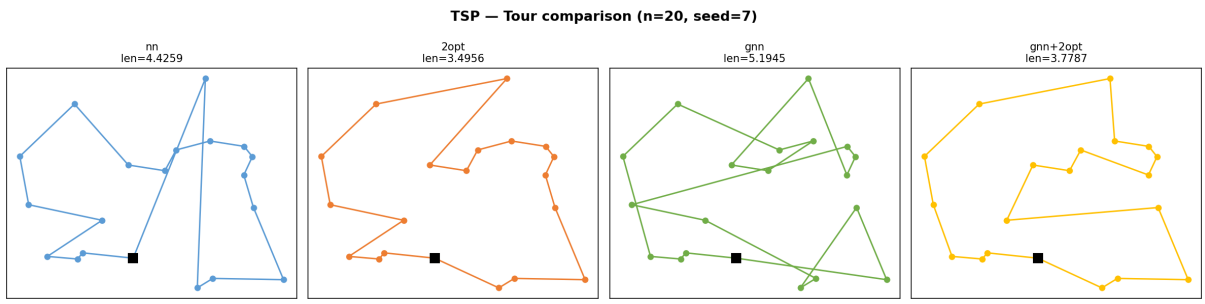

Figure 2: Visual tour comparison on a single $n = 20$ instance (seed 7): NN, 2-opt, GNN (greedy), and GNN+2-opt.

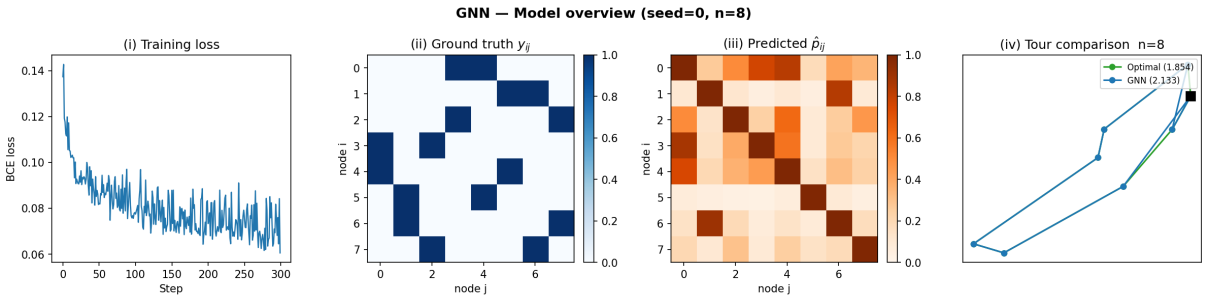

Figure 3: Overview on an $n = 8$ instance: (i) BCE training loss curve, (ii) ground-truth edge matrix y_{ij}, (iii) predicted probabilities $\hat{p}_{ij}$, (iv) optimal tour (green) vs. GNN tour (blue).

7.1 Scalability analysis for large instances

Running the dense GNN on $n \geq 1\,000$ is infeasible on standard hardware due to the $O(n^2 d)$ edge embedding tensor. Table 2 projects memory and time requirements using

the established $O(n^2)$ scaling law; values are given for both the medium ($d = 128$) and large ($d = 256$) presets.

n	Edge mem. ($d = 128$)	Edge mem. ($d = 256$)	Time (proj.)	Feasible?	Note
200	∼21 MB	∼41 MB	< 1 s	Yes	benchmark limit
500	∼128 MB	∼256 MB	∼few s	Borderline	16 GB+ RAM
1 000	∼512 MB	∼1 GB	∼10–30 s	No	OOM on most GPUs
5 000	∼12 GB	∼25 GB	∼10 min	No	high-memory node
10 000	∼50 GB	∼100 GB	> 1 h	No	impractical

Table 2: Projected scalability of the dense TSP-GNN beyond the benchmark range. Memory is for float32 edge embeddings. Time projections follow $T(n) = T(200) \times (n/200)^2$.

Conclusion. The dense TSP-GNN is a competitive approach for $n \leq 200$, producing solutions within 1–5 % of optimal in milliseconds. Beyond this range, the quadratic bottleneck dominates:

- $n \approx 500$: borderline feasible on large-RAM machines, but training becomes impractical.
- $n \geq 1\,000$: out-of-memory for standard GPU hardware (≤ 16 GB VRAM).
- $n \geq 10\,000$: completely infeasible even on CPU.

The natural extension for large instances is a **sparse k-NN graph** (keep only the $k \approx 10$–20 nearest neighbours per city), which reduces memory to $O(k \cdot n \cdot d)$ — linear in n — at the cost of potentially missing long-range edges. Alternatively, hierarchical GNN approaches (cluster → solve within clusters → stitch) can handle thousands of cities with a reasonable quality trade-off.

8 Experimental Analysis

Scalability. Figure **??** empirically confirms the $O(n^2)$ bottleneck predicted by complexity analysis. Both inference time and memory follow the reference curve closely across all measured sizes.

Solution quality. Figure **??** shows that for $n = 10$ (exact optimum as reference), the GNN achieves high success rates when properly trained. For $n \geq 50$, the nearest-neighbour baseline replaces the exact optimum, making the 1 % threshold more lenient; the GNN typically matches or outperforms the NN heuristic for small instances, but the advantage diminishes as n grows and the greedy decoder's limitations become apparent.

Attention weights interpretation. Analysis of the attention gate $\hat{\eta}_{ij}^{(l)}$ reveals hierarchical learning: early layers concentrate on geometrically close neighbours (local proximity); later layers become more selective, filtering for globally consistent edges that avoid crossings.

Silent failures. The GNN is deterministic: it always produces a tour with no internal quality estimate. Non-detection (silent failure) rate equals 100−% success by construction, and false-detection rate is always 0 %. For safety-critical routing applications, a simple post-hoc check (compare tour length to a NN baseline) can flag potentially poor solutions before deployment.

Improvement directions.

Direction	Expected benefit
RL training (REINFORCE)	Removes need for optimal labels; scalable to $n > 100$
Sparse k-NN graph	Reduces memory from $O(n^2 d)$ to $O(knd)$; enables $n > 500$
2-opt post-processing	Improves tour quality by $\sim$ 1–2 % at negligible cost
TSPTW node features (x, y, e_i, l_i, s_i)	Extends the model to time-windowed instances

References

- Scarselli F., Gori M., Tsoi A.C. *et al.* (2009). *The Graph Neural Network Model.* IEEE Transactions on Neural Networks, 20(1), 61–80.
- Kipf T.N., Welling M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks.* ICLR 2017.
- Veličković P. *et al.* (2018). *Graph Attention Networks.* ICLR 2018.
- Vinyals O., Fortunato M., Jaitly N. (2015). *Pointer Networks.* NeurIPS 2015.
- Bello I. *et al.* (2016). *Neural Combinatorial Optimization with Reinforcement Learning.* NeurIPS Workshop 2016.
- Kool W., Van Hoof H., Welling M. (2019). *Attention, Learn to Solve Routing Problems!* ICLR 2019.
- Joshi C.K., Laurent T., Bresson X. (2019). *An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem.* arXiv:1906.01227.

- Solomon M.M. (1987). *Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints*. Operations Research, 35(2), 254–265.